

Lecture 6

Functions Miscellaneous

Today Lecture Contents

- **Passing Arrays to Functions**
- **Returning values by value,**
- **Returning values by reference**
- **Returning values by address**
- **Inline Functions**
- **Function Overloading**

Brief Revision: Function Definition

- Define *function header* and *function body*
- Value-returning functions

```
return-data-type function-name(parameter list)
{
    constant declarations
    variable declarations

    other C++ statements

    return value
}
```

- Non value-returning functions

```
void function-name(parameter list)
{
    constant declarations
    variable declarations

    other C++ statements
}
```

Passing Arrays to functions

- Arrays are passed by pointer.
- The “[]” in the formal [parameter specification](#) indicates that the variable is an array.
- It is a good practice to pass the dimension of the array as another parameter.

Passing 1D Array Example

Notice empty brackets



```
int ListMinimum(int Ar[], int asize) {  
    int SmallestValueSoFar = Ar[0];  
    for (int i = 1; i < asize; ++i) {  
        if (Ar[i] < SmallestValueSoFar) {  
            SmallestValueSoFar = Ar[i];  
        }  
    }  
    return SmallestValueSoFar ;  
}
```

As Number is a pointer to the first element of the array. So we can pass Number to the function ListMinimum() without using & operator.

```
int main() {  
    int Number[6] ={3, 88, 7, 9, 1, 24};  
    cout << ListMinimum(Number, 6) << endl  
  
    return 0;  
}
```

Passing 2D array to function

- You can pass a two-dimensional array to a function;
- however, C++ requires that the column size to be specified in the function declaration.
- Example given on next slide uses a function that sum up two two-dimensional array into a third one.
 - All the three 2d arrays are passed to function.
 - Note that any change in the array in function is reflected in the main program.

```

int main() {
int max_rows = 2;
int max_cols = 5;

int a[max_rows][max_cols] = {{1, 2, 3, 5, 6},
                               {1, 2, 3, 5, 6}},
    b[max_rows][max_cols] = {{2, 2, 4, 3, 7},
                               {1, 0, 1, 3, 4}},
    c[max_rows][max_cols];

    int i, j;
    add_array(a, b, c, max_rows);
cout << "Content of array c is: \n";
    for (i = 0; i < max_rows; i++){
        for (j=0; j < max_cols; j++)
            cout << c[i][j] << ", ";
        cout << endl;
    }
    return 0;
}

```

Example: Passing 2d arrays to functions

// Sum up two 2-dimensional arrays into a third one

```
#include <iostream>
```

```
using namespace std;
```

// c[i][j] = a[i][j] + b[i][j]

```
void add_array(int a[][max_cols],  
               int b[][max_cols],  
               int c[][max_cols],  
               int rows)
```

```
{    int i, j;  
    for (i=0; i < rows; i++)  
        for (j=0; j < max_cols; j++)  
            c[i][j] = a[i][j] + b[i][j];  
}
```


RETURNING FROM FUNCTIONS

Return by value

- Return by value is the simplest and safest return type to use.
- When a value is returned by value, a copy of that value is returned to the caller.
- As with pass by value, you can return by value literals (e.g. 5), variables (e.g. x), or expressions (e.g. x+1), which makes return by value very flexible.
- Another advantage of return by value is that you can return variables (or expressions) that involve local variables declared within the function without having to worry about scoping issues.

How to use 'Return by value'

- Return by value is the most appropriate when returning variables that were declared inside the function, or for returning function arguments that were passed by value.
- However, like pass by value, return by value is slow for structs and large classes.

```
int doubleValue(int x)
{
    int value = x * 2 ;
    return value; // A copy of value will be returned here
} // value goes out of scope here
```

When to Use/Not use Return by value

- When to use return by value:
 - When returning variables that were declared inside the function
 - When returning function arguments that were passed by value
- When not to use return by value:
 - When returning a built-in array or pointer (use return by address)
 - When returning a large struct or class (use return by reference)

Return by address

- Returning by address involves returning the address of a variable to the caller.
- Similar to pass by address, return by address can only return the address of a variable, not a literal or an expression (which don't have addresses). by value, a copy of that value is returned to the caller.
- return by address is fast because return by address just copies an address from the function to the caller,

Use of 'Return by Address'

- Return by address is often used to return dynamically allocated memory to the caller:

```
int* allocateArray(int size)
```

```
{  
    return new int[size];           //creates an array dynamically and returns its address  
}
```

```
int main()
```

```
{  
    int *array = allocateArray(25);  
  
    // do stuff with array  
  
    delete[] array;  
    return 0;  
}
```

Downside of using 'Return by address'

- if you try to return the address of a variable local to the function, your program will exhibit undefined behavior.
- Because the local variable will go out of scope as soon as the function terminates
- the caller ends up with the address of non-allocated memory (a dangling pointer),

```
int* doubleValue(int x)
{
    int value = x * 2;
    return &value; // return value by address here
} // value destroyed here
```

When to use/not use return by address:

- When to use return by address:
 - When returning dynamically allocated memory
 - When returning function arguments that were passed by address
- When not to use return by address:
 - When returning variables that were declared inside the function or parameters that were passed by value (use return by value)
 - When returning a large struct or class that was passed by reference (use return by reference)

Return by Reference

- Similar to return by address, values returned by reference must be variables
 - you should not return a reference to a literal or an expression that resolves to a temporary value, as those will go out of scope at the end of the function and you'll end up returning a dangling reference.
- When a variable is returned by reference, a reference to the variable is passed back to the caller. The caller can then use this reference to continue modifying the variable
- Return by reference is also fast, which can be useful when returning structs and classes (Structs and Classes will be covered in future lessons).

How to use 'return by reference'

```
#include <iostream>
```

```
Using namespace std;
```

```
int n;
```

```
int& set();      //Function Prototype
```

```
int main()      //main function
```

```
{
```

```
    set() = 5;
```

```
    cout<<n;
```

```
    return 0;    }
```

```
int& set()      {      //function definition  
    return n;    //returns n by reference  
}
```

Downside of using return by reference

- Similar to return by address, local variables should not be returned by reference. For example:

```
int& doubleValue(int x)
{
    int value{ x * 2 };
    return value; // return a reference to value here
} // value is destroyed here
```

- the program returns a reference to a value that is destroyed when the function returns. Hence the caller receives a reference to garbage.

When to use/not use return by reference:

- When to use return by reference:
 - When returning a reference parameter
 - When returning a member of an object that was passed into the function by reference or address
 - When returning a large struct or class that will not be destroyed at the end of the function (e.g. one that was passed in by reference)
- When not to use return by reference:
 - When returning variables that were declared inside the function or parameters that were passed by value (use return by value)
 - When returning a built-in array or pointer value (use return by address)

INLINE FUNCTIONS

Inline functions

- Benefits of using Functions
 - The code inside the function can be reused.
 - Code reuse is good. The code in the function can be changed only once rather than the changing each instance of the code which becomes duplicate and error prone.
 - Code becomes easy to read and understand.
 - It is easy to debug functions
 - Functions provide type checking to ensure function call arguments match the function parameters (function-like macros don't do this, which can lead to errors).
- However, function call causes performance overhead.
 - For example, for small functions the time needed to make the function call is often a lot more than the time needed to actually execute the function's code.
- Therefore, code written in-place is significantly faster.
 - C++ offers a way to combine the advantages of functions with the speed of code written in-place: **inline functions**.

Inline Functions

- The keyword `inline` is used to request that the compiler treat your function as an inline function.
- When the compiler compiles your code, all inline functions are expanded in-place -- that is, the function call is replaced with a copy of the contents of the function itself, which removes the **function call overhead**!
- The **downside** is that because the inline function is expanded in-place for every function call, this can make your compiled code quite a bit larger, especially if the **inline function** is long and/or there are many calls to the inline function.

Inline Functions

- Consider the following code →
- The function max is called twice.
- Each time a one line max function is called it causes significant performance overhead.
- Max can be made inline. As below:

```
inline int min(int x, int y)
{
    return x > y ? y : x;
}
```

- The resulting main code becomes:

```
int max(int x, int y)
{
    return x > y ? x : y;
}
```

```
int main()
{
    std::cout << max(5, 6) << '\n';
    std::cout << max(3, 2) << '\n';
    return 0;
}
```

```
int main()
{
    std::cout << (5 > 6 ? 6 : 5) << '\n';
    std::cout << (3 > 2 ? 2 : 3) << '\n';
    return 0;
}
```


Inline Functions

- Code with inline functions execute quite a bit faster, at the cost of the compiled code being slightly larger.
- But inline functions are suitable for shorter functions, e.g., no more than few lines.
- The keyword inline is only a recommendation, and the compiler may ignore the request particularly if the function is larger.

Inline functions: Another Example

- Write a function to compute the distance between two points (x_1, y_1) and (x_2, y_2)

```
inline double distance (double x1, double y1,  
                        double x2, double y2)  
{  
    return sqrt(pow(x1-x2,2)+pow(y1-y2,2));  
}
```

FUNCTION OVERLOADING

Function Overloading

- Function overloading

- Functions with same name and different parameters

- Should perform similar tasks

- I.e., function to square `ints` and function to square `floats`

- ```
int square(int x) {return x * x;}
```

- ```
float square(float x) { return x * x; }
```

- Overloaded functions distinguished by signature

- Based on name and parameter types (order matters)

**fig03_25.cpp**
(1 of 2)

```
1 // Fig. 3.25: fig03_25.cpp
2 // Using overloaded functions.
3 #include <iostream>
4
5 using std::cout;
6 using std::endl;
7
8 // function square for int values
9 int square( int x )
10 {
11     cout << "Called square with int argument: " << x << endl;
12     return x * x;
13 }
14 // end int version of function square
15
16 // function square for double values
17 double square( double y )
18 {
19     cout << "Called square with double argument: " << y << endl;
20     return y * y;
21 }
22 // end double version of function square
23
```

Overloaded functions have the same name, but the different parameters distinguish them.



fig03_25.cpp
(2 of 2)

fig03_25.cpp
output (1 of 1)

```
24 int main()
25 {
26     int intResult = square( 7 );           // calls int version
27     double doubleResult = square( 7.5 ); // calls double version
28
29     cout << "\nThe square of integer 7 is 49\n";
30     cout << "\nThe square of double 7.5 is 56.25\n";
31     cout << endl;
32
33     return 0; // indicates successful termination
34
35 } // end main
```

The proper function is called
based upon the argument
(**int** or **double**).

Called square with int argument: 7
Called square with double argument: 7.5

The square of integer 7 is 49
The square of double 7.5 is 56.25

Function Overloading

- C++ provides the capability of using the same function name for more than one function (*function overloading*).
- The compiler must be able to determine which function to use based on the **number** and **data types** of the parameters.

Function Overloading (cont.)

```
void cdabs(int x)
{
    if (x<0)
        x = -x;
    cout << "The abs value of the integer is " << x << endl;
}
```

```
void cdabs(float x)
{
    if (x<0)
        x = -x;
    cout << "The abs value of the float is " << x << endl;
}
```

- **Warning:** creating overloaded functions with identical parameter lists and **different return types** is a syntax error !!

line